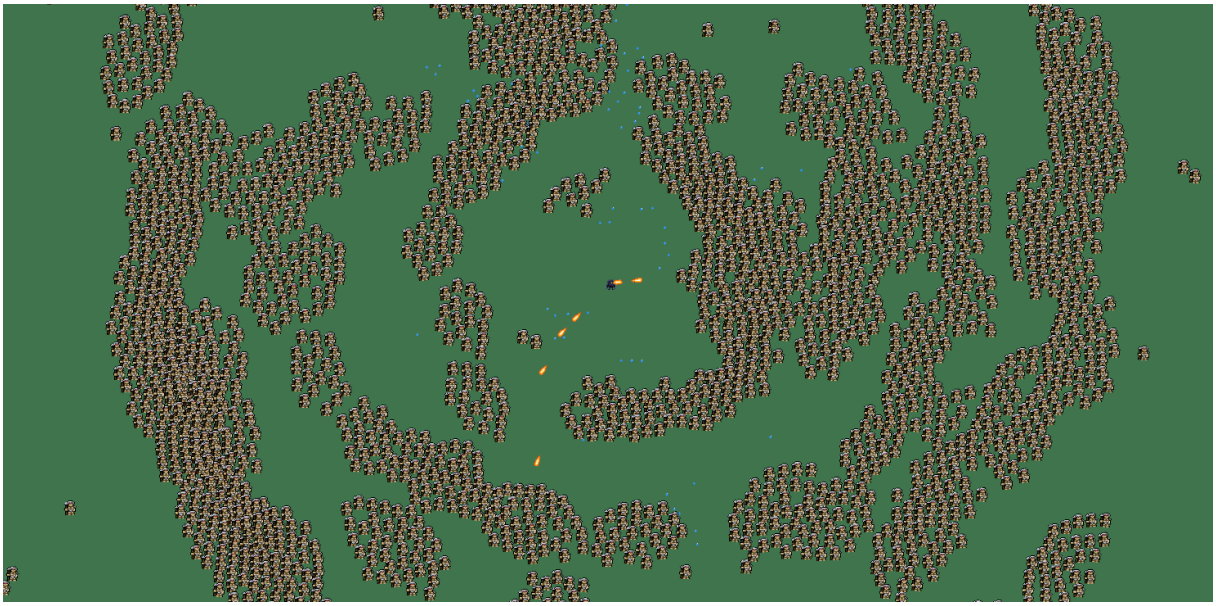
Roguelike: Performance Maxxing



By: Martin Simon
Student in Software Engineering

Table of contents

1. Context & Objectives 3
   1.1 Context 3
   1.2 Objectives 4
2. OOP 4
   2.1 OOP Implementation Choices 4
   2.2 Class diagram 6
   2.3 Performance analysis 7
      2.3.1 | 2000 Enemies 9
      2.3.2 | 4000 Enemies 9
      2.3.3 | 6000 Enemies 10
      2.3.4 | 8000 Enemies 10
      2.3.5 | 10000 Enemies 11
   2.4 Performance Summary table 11
   2.5 Memory Management & Garbage Collection 11
   2.6 Conclusion 12
3. DOTS/ECS 13
   3.1 DOTS/ECS Phase Overview 13
   3.2 DOTS/ECS Implementation Choices 13
   3.3 Performance analysis 14
      3.3.1 | 2,000 Enemies 15
      3.3.2 | 4,000 Enemies 15
      3.3.3 | 6,000 Enemies 15
      3.3.4 | 8,000 Enemies 16
      3.3.5 | 10,000 Enemies 16
      3.3.6 | 50,000 Enemies 17
      3.3.7 | 100,000 Enemies 17
      3.3.8 | 250,000 Enemies 17
      3.3.9 | 500,000 Enemies 18
      3.3.10 | 1,000,000 Enemies 18
   3.4 Performance Summary table 18
   3.5 OOP vs DOTS/ECS Comparison 19
   3.6 Multithreading in Action 20
   3.7 Memory Management & Garbage Collection 21
   3.8 Conclusion 21

4. Profiling-Driven Optimization 23
4.1 Starting Point 23
4.2 Step A: Removing the Synchronization Point 23
4.3 Step B: A Sorted Spatial Grid 25
4.3.1 | First Version: a Sequential Sort 25
4.3.2 | Second Version: a Parallel Sort 26
4.4 Results and the New Bottleneck 27
4.5 Conclusion 28

1. Context & Objectives

1.1 Context

As a lifelong video game enthusiast, I have recently transitioned from player to creator during my free time. This project is the combination of my passion for gaming and my academic background as a Software Engineering student.
The core motivation behind this development is to apply rigorous software engineering principles, such as modular architecture, clean code, and system optimization, to the complex environment of game development. By building a functional Roguelike from scratch, I aim to demonstrate how engineering

patterns can solve the unique performance challenges inherent in "Survivor-like" games, where thousands of entities must be managed simultaneously.

Note that the main purpose of this project is not to make the most entertaining and fun game at first, I want to make a powerful game that could be played on almost any computer. I intend to implement fun features and make the game enjoyable after ending this project, follow my LinkedIn to see any potential announcement about the advancement of the game.

1.2 Objectives

The objective of this project is to fully compare the performances of a roguelike game using two fundamentally different architectures: Object-Oriented Programming (OOP) and DOTS/ECS.

- OOP Phase: The purpose is to find the limit where the player can have the most optimized and stable experience using standard Unity tools.
- DOTS/ECS Phase: We will then measure the performance gains with the new implementation and see how far we can push the limits of the game.

2. OOP

2.1 OOP Implementation Choices

Spatial grid: Reduced complexity from $O(n^2)$ to $O(1)$. To check collision of enemies with other enemies, you can just look at the cells around it.

Chunks: To make sure you can properly dezoom and move freely on the map, I implemented a chunk system. The chunks are loaded at the beginning and move with your character. As the game dezooms, it adds more chunks if needed to prevent the player from seeing outside the map.

Pools: Pools are used on entities that appear a lot on the screen, such as projectiles, enemies and experience. The pools allow you to allocate the necessary memory when the game starts and greatly

reduces creation/destruction costs for each frame.

Observer: Observer Pattern on player to notify UI to update when necessary.

WeaponManager/ Scriptable Objects: WeaponManager handles the whole weapon logic, I added an interface IWeapon that every weapon inherits from, WeaponManager only has to call IWeapon.Attack() for each weapon in the player's inventory. The weapons are also ScriptableObjects and not MonoBehaviour, this allows you to add new weapons very efficiently with only a few modifications only in the Unity Interface.

Handle massive enemy horde movements (Boids Algorithm): To manage thousands of enemies on screen without severe jittering or overlapping, I implemented a custom Steering Behavior inspired by the Boids algorithm. Instead of strictly applying instantaneous repulsive forces that cause visual "teleportation" and physics glitches, the enemies compute a blended directional vector. This blends a "Seek" behavior (moving toward the player) with a weighted "Separation" behavior (avoiding nearby neighbors). The separation vector is smoothly interpolated over time (Vector3.Lerp) to ensure a fluid, liquid-like horde movement. Additionally, the heavy separation calculations are time-sliced and distributed across multiple frames to significantly reduce CPU load.

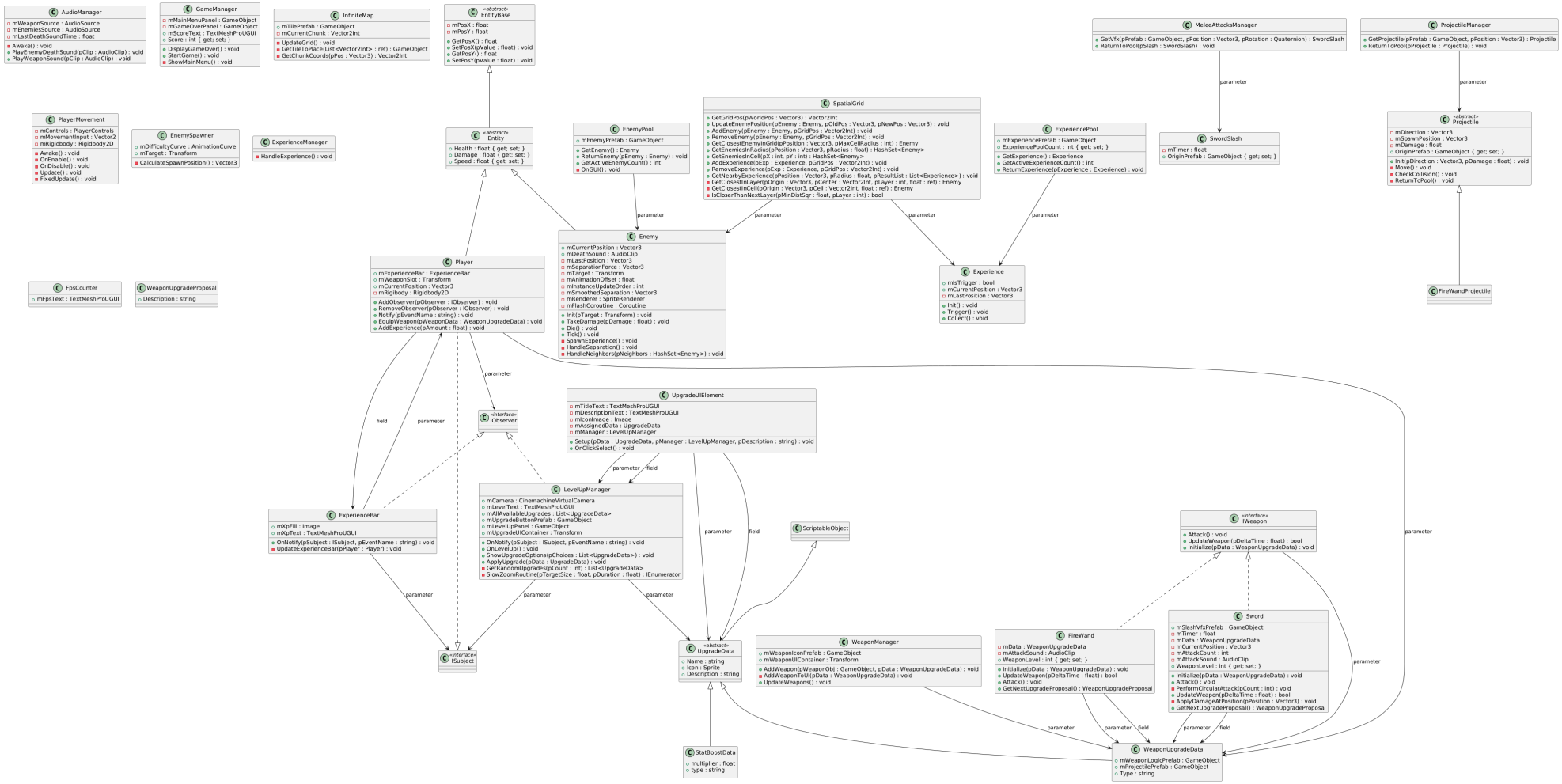
Clean Architecture:
Throughout the OOP phase, I made several adjustments to keep the architecture simple and the workflow efficient:

- **Removing Redundant Classes:** I realized that a custom EntityBase class to handle movements was unnecessary since Unity's built-in Transform component already handles positioning perfectly. Removing it simplified the entity hierarchy. I still kept it for the main attributes such as Health, Speed, Damage, etc…

- **Inspector Clean-up:** I systematically added [SerializeField] attributes to properly link UI elements directly within the Unity Inspector. This cleared up console warnings and made tweaking variables much easier.

- **Dynamic Item Drops:** I evolved the game design regarding loot. Instead of restricting items to spawn on fixed grid tiles, I modified the logic so they simply drop on the ground as physical objects, making the gameplay feel more dynamic.

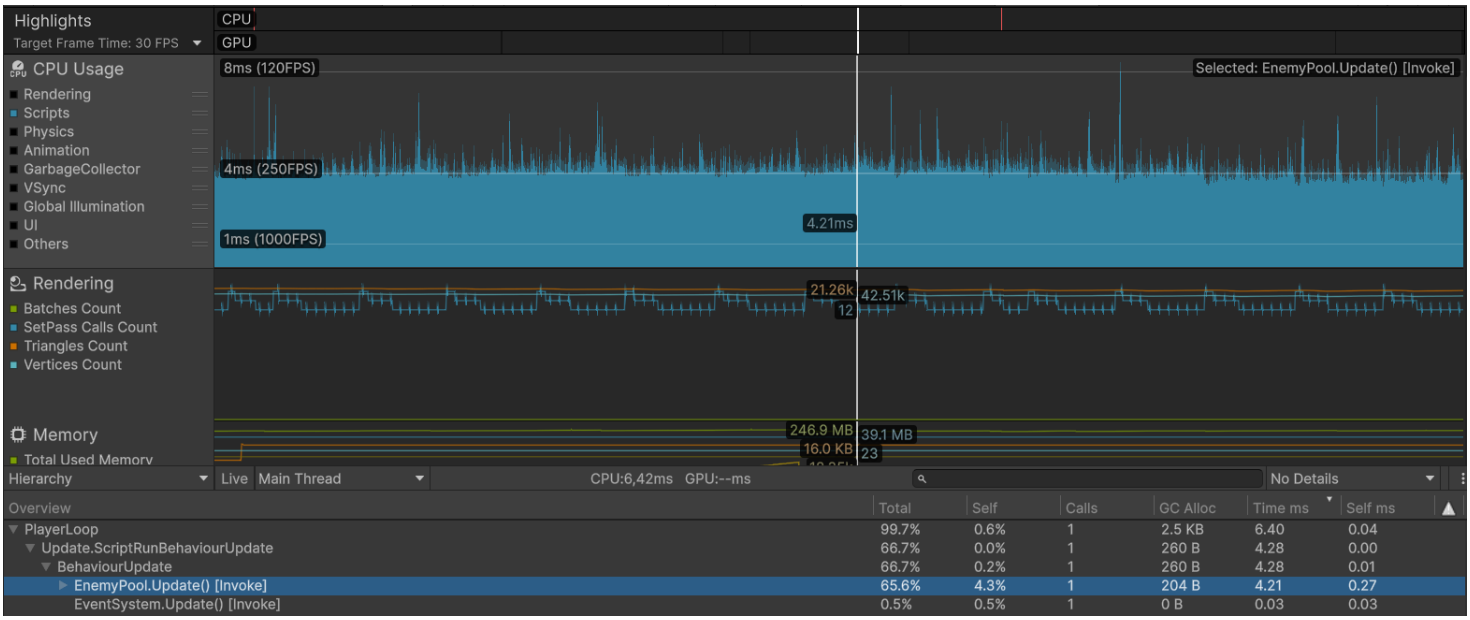
2.2 Class diagram



2.3 Performance analysis

Let's now try to find the limits of the OOP implementation. To begin with, I will analyze the code and try to find out what reduces the performance the most.

After using the profiler we see that the EnemyPool.Update() is what makes us lose the most performance:



We know the Update in EnemyPool calls the Tick () method in Enemy.cs:

```
void Update()
{
    for (int i = 0; i < mActiveEnemiesList.Count; i++)
    {
        mActiveEnemiesList[i].Tick();
    }
}
```

Now we know where to look at, let's go in the Tick () method and make a few changes at different parts so we can have more information in the profiler:

```
Profiler.BeginSample("1_Enemy_Maths_Boids");

mCurrentPosition = transform.position;
var lTargetPosition = mTarget.position;

// Compute the base direction towards the player
Vector3 lTargetDirection = (lTargetPosition - mCurrentPosition).normalized;

if ((Time.frameCount + mInstanceUpdateOrder) % 4 == 0) // Spread separation calculations over 4 frames to reduce CPU load
{
    HandleSeparation();
}

// Interpolate the separation force to prevent jittering and teleporting effects
mSmoothedSeparation = Vector3.Lerp(mSmoothedSeparation, mSeparationForce, Time.deltaTime * 10f);

// Combine target seeking and separation.
// Separation is weighted heavily (x4.0) to prioritize avoiding neighbors and prevent clumping
Vector3 lFinalDirection = (lTargetDirection + (mSmoothedSeparation * 4.0f)).normalized;

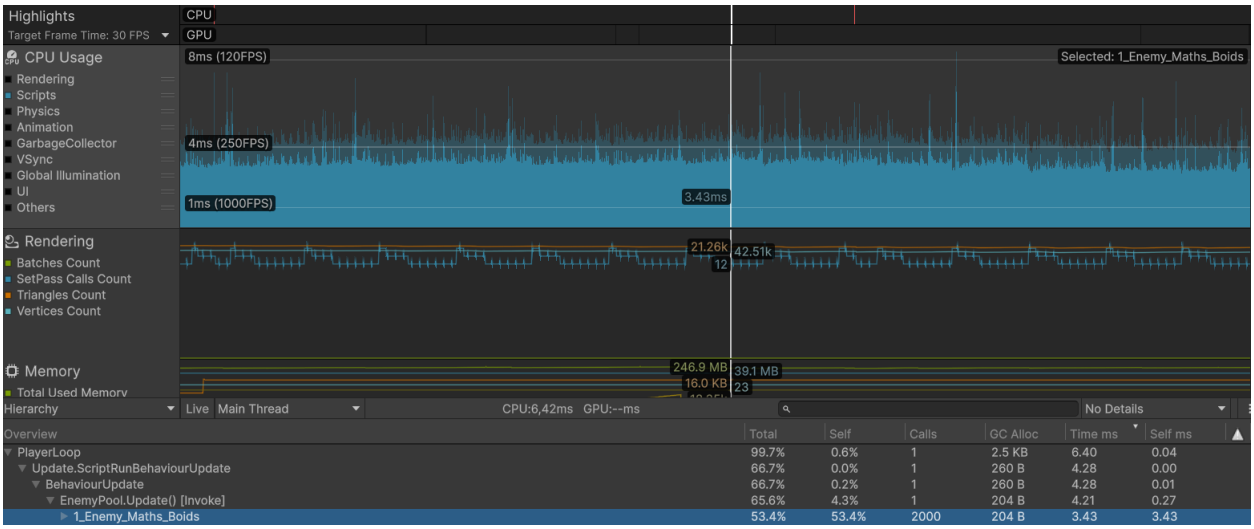
// Apply movement using the strict base speed to maintain a steady, fluid horde pace
mCurrentPosition += Speed * Time.deltaTime * lFinalDirection;

// Update the enemy's position within the spatial grid for accurate neighbor detection
SpatialGrid.Instance.UpdateEnemyPosition(this, mLastPosition, mCurrentPosition);
mLastPosition = mCurrentPosition;

// Apply a procedural tilt effect based on time to animate the sprite
float lTilt = Mathf.Sin((Time.time + mAnimationOffset) * mTiltSpeed) * mTiltAngle;

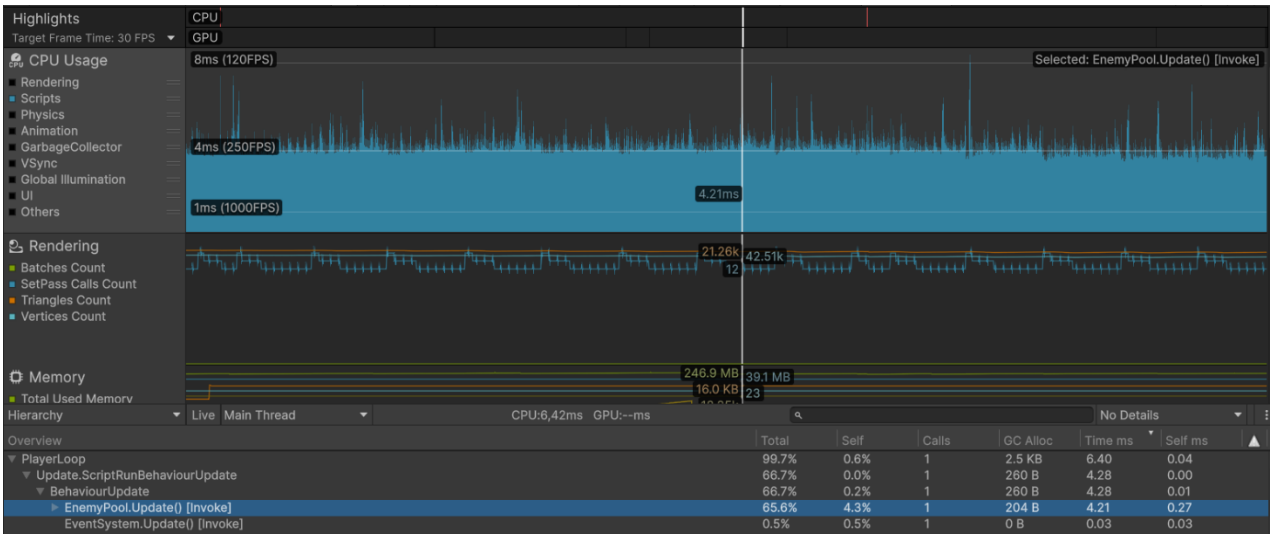
Profiler.EndSample();
```

Now let's go back to the profiler. We can now identify where the issue comes from. To no surprise, it comes from the Boids Algorithm, this is the part of the code that makes the most computations.



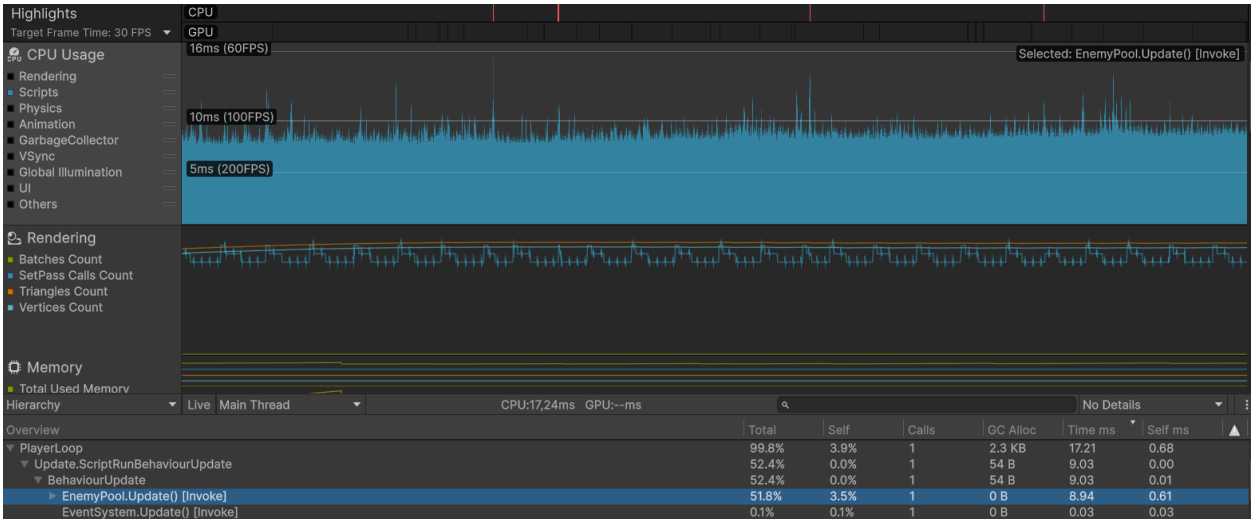
Let's now do a series of tests to see where the limit of this implementation is.

2.3.1 | 2000 Enemies



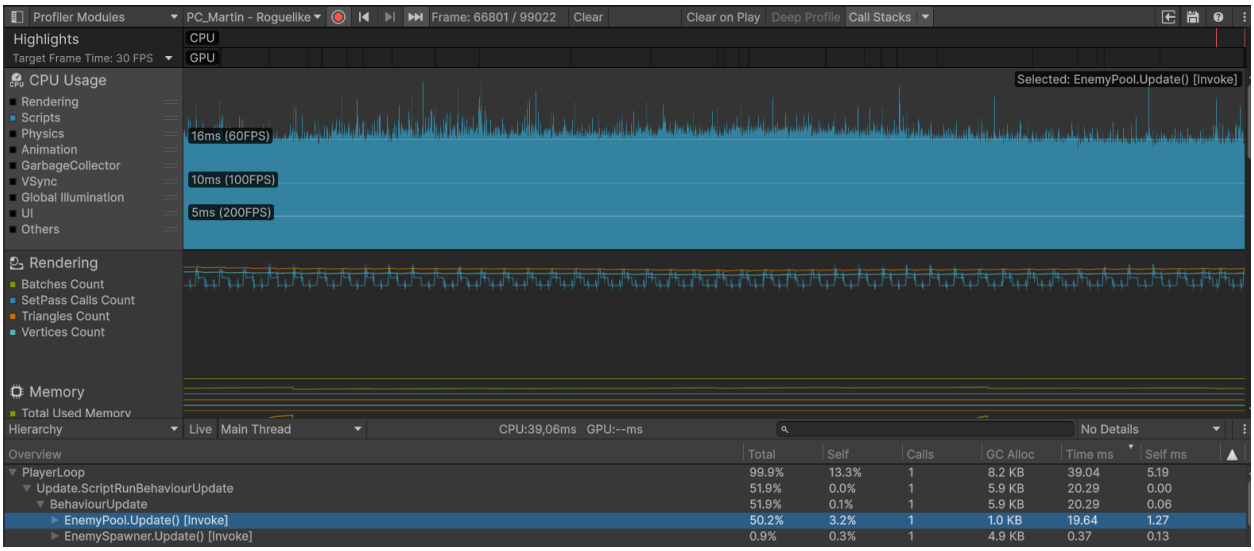
With 2000 enemies, we can see the update only takes 4.21ms and the in-game FPS is very stable above 144 fps.

2.3.2 | 4000 Enemies



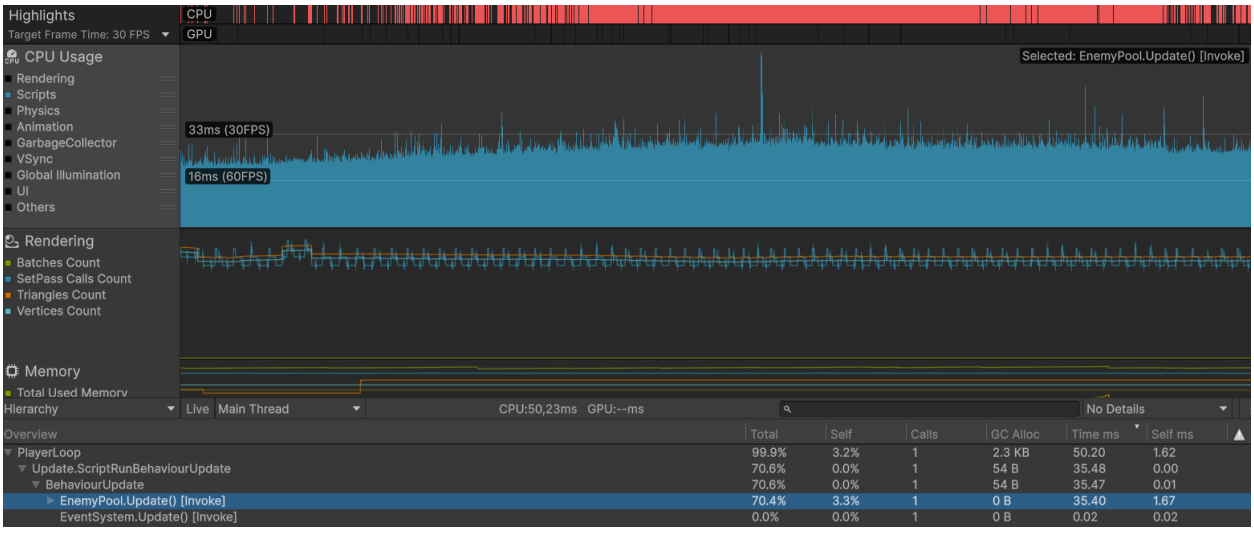
With 4000 enemies, we can already notice an important loss of performance, the method's time to compute more than doubled, going from 4.21ms to 8.94ms. In-game, the FPS starts dropping slightly below 144 fps, around 120 fps.

2.3.3 | 6000 Enemies



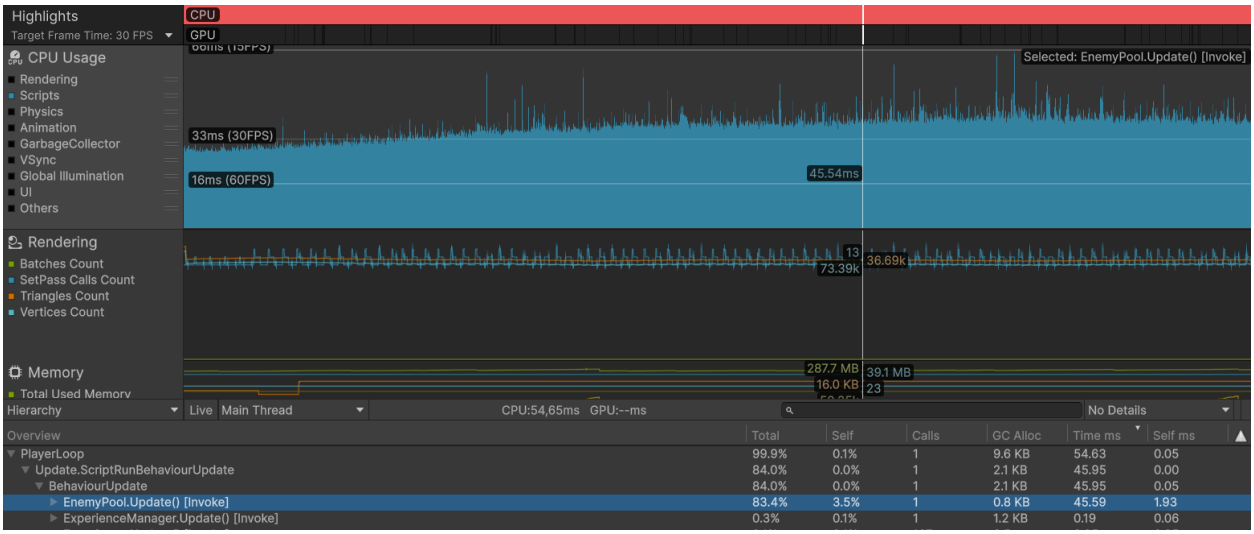
With 6000 enemies, the execution time now takes 19.64ms, the FPS in-game is around 60 which is an acceptable number. The player would probably still be able to play and not be too annoyed by the frame drop due to the high number of things displayed on the screen. This is a sort of limit where it's accepted by the player as he knows that a lot of instances reduce performance.

2.3.4 | 8000 Enemies



With 8000 enemies, it now takes 35.40ms, the game is barely playable with 30 FPS, and the player would probably still accept that if it is **temporary.** We are now close to the limit and we can predict that the next test will be the last as the game might become unplayable.

2.3.5 | 10000 Enemies



To update 10000 Enemies, it now takes 45.59ms, the in-game FPS is now below 20 FPS which makes it unplayable as expected. We can say that 10000 enemies is **the limit for the OOP implementation.**

2.4 Performance Summary table

	2000 Enemies	4000 Enemies	6000 Enemies	8000 Enemies	10000 Enemies
Execution time (ms)	4,21	8,94	19,64	35,4	45,59
Scaling Factor	0	2,12	2,20	1,80	1,29
Time per Enemy (ms)	0,0021	0,0022	0,0033	0,0044	0,0046

In a Survivor-like game, the player's tolerance for performance drops is implicitly linked to the visual intensity on screen. At a scale of 10,000 active enemies, the system maintains an execution time of 45.59 ms. While this exceeds the ideal 16.6 ms window for 60 FPS, this drop to approximately 20 FPS marks the functional limit of the OOP implementation.

The Time per Enemy remains highly efficient, only increasing from 0.0021 ms to 0.0046 ms as the horde grows. This linear scalability, maintained by a custom Boids-inspired steering behavior and a spatial grid system, ensures that the performance remains predictable.

2.5 Memory Management & Garbage Collection

Optimization is not solely about CPU time; it also requires strict memory monitoring. Implementing the Object Pool pattern drastically reduces dynamic allocations within the heaviest game loops. As demonstrated by the profiler, our critical path for enemy updates generates an extremely low allocation

of roughly **0 - 1KB** per frame. At this scale, this micro-allocation is negligible. Its impact on Garbage Collector triggers is practically non-existent, ensuring a smooth experience free from stuttering or lag spikes. Isolating and measuring this behavior validates the stability of the OOP architecture. Furthermore, it proves that the performance bottleneck encountered at 10,000 entities is primarily due to memory fragmentation rather than excessive Garbage Collection overhead.

2.6 Conclusion

This first phase successfully pushed standard Unity tools and classic OOP architecture to their absolute limits. By applying strict engineering principles, such as a spatial grid, object pooling, optimized Boids algorithms, and clean memory management, the game maintains a playable framerate with a massive horde of 8,000 active entities. However, a clear technical ceiling is reached at 10,000 entities, where the framerate drops to approximately 20 FPS.

This is, I believe, a robust and optimized result for an OOP project. Yet, to meet high-level production standards, where low-level optimization and massive scale are critical, this approach reveals its fundamental hardware limitations. The underlying issue is not the algorithmic logic itself, but how OOP inherently scatters data across the RAM. This memory fragmentation leads to constant Cache Misses, starving the CPU and preventing it from computing at its full potential.

To break this performance ceiling, drop below the 16.6 ms frame time at a massive scale, and truly maximize hardware efficiency, a paradigm shift is required. We must abandon the object-oriented logic in favor of a Data-Oriented approach that aligns with how the CPU cache operates. This is the exact purpose of the upcoming DOTS/ECS implementation.

3. DOTS/ECS

3.1 DOTS/ECS Phase Overview

After reaching the limits of the OOP implementation at around 10,000 enemies, the second phase of this project re-implements the enemy simulation using Unity's Data-Oriented Technology Stack (DOTS): Entities (ECS), the C# Job System and the Burst Compiler.

In ECS, an enemy is no longer a GameObject with MonoBehaviours, but an Entity: a simple identifier linked to a set of components that only contain data (position, speed, damage...). Entities sharing the same set of components are stored contiguously in 16 KB memory blocks called archetype chunks. The logic lives in Systems that read these arrays linearly, which is exactly how the CPU cache prefers to access memory.

The Job System then splits this work across all CPU cores, and Burst compiles the jobs into highly optimized native code. Together, these three tools target the exact root cause identified in the OOP phase: data scattered in memory and a simulation running on a single thread.

I chose a hybrid approach: only the massive part of the game (enemies and experience orbs) was moved to ECS, while the player, the weapons and the UI remain classic MonoBehaviours. Converting objects that only exist once would bring no measurable benefit, and would make the gameplay code harder to iterate on.

3.2 DOTS/ECS Implementation Choices

Spatial hash grid (EnemyHashSystem): The spatial grid from the OOP phase was rebuilt as a NativeParallelMultiHashMap, filled every frame by a parallel Burst job. Its capacity grows with the horde and shrinks back when the number of enemies drops sharply, which avoids a native reallocation every frame.

Parallel horde movement (EnemyMovementSystem): The same Seek + Separation steering behavior as in the OOP version, now executed as a Burst-compiled IJobEntity on all worker threads. The optimizations from the OOP phase were kept: the separation is time-sliced over 4 frames, limited to 4 neighbors, and the neighboring cells are visited in a spiral order to avoid any directional bias in the horde.

Entity Command Buffers: Creating or destroying entities (structural changes) is not allowed inside parallel jobs. Enemy deaths and experience orb spawns are therefore recorded in Entity Command Buffers and played back once, at the end of the simulation.

GPU-driven rendering: Enemies are rendered by Entities Graphics, which relies on the BatchRendererGroup API to draw hundreds of thousands of instances in a few dozen batches.

EnemyBridge facade: Weapons and projectiles remain MonoBehaviours and never use the ECS API directly. They go through a static facade, EnemyBridge, exposing simple methods such as TryGetClosestEnemy() and DealDamage(). The spatial grid and the command buffers stay hidden from the gameplay code.

Safe enemy destruction: Since command buffers are only played back at the end of the frame, an enemy hit by two projectiles during the same frame could be destroyed twice, dropping two experience orbs and throwing an error during playback. Each enemy now carries an enableable EnemyDeadTag component, switched on as soon as its destruction is queued, so that every code path (weapons, contact with the player, spawner) skips enemies that are already dying.

In-game benchmark controller: A small HUD and keyboard shortcuts allow changing the target horde size at runtime, from 1,000 up to 1,000,000 enemies, without recompiling or restarting the game.

3.3 Performance analysis

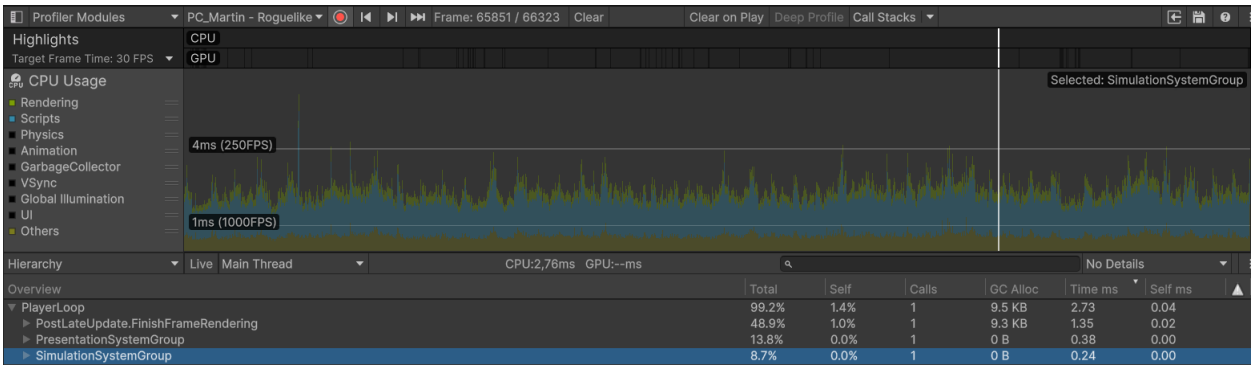
To compare both implementations fairly, I measured the equivalent of EnemyPool.Update() from the OOP phase. In ECS, the enemy logic is split across several systems that all run inside the SimulationSystemGroup, so this group is the metric used below.

There is one important difference: the heavy work no longer runs on the main thread but on the worker threads. The time shown for SimulationSystemGroup on the main thread therefore includes scheduling the jobs and waiting for them to complete. It also includes the experience orbs and the spawner. This makes it a slightly pessimistic, but honest, equivalent of the OOP measurement.

I also report the PlayerLoop time, which represents the whole CPU frame of the game (simulation, rendering preparation, UI...).

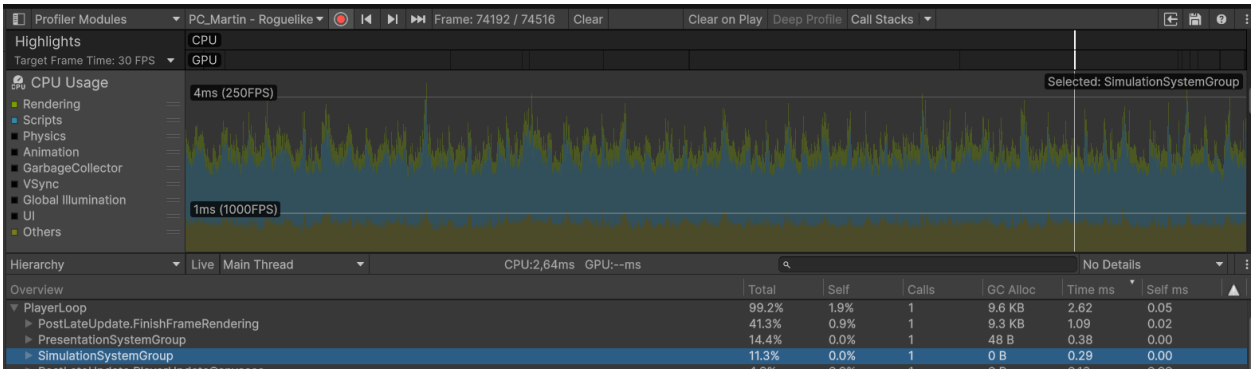
Both implementations were measured on the same machine (HP Victus 15 laptop: Intel Core i5-12450H with 8 cores and 12 threads, NVIDIA GeForce RTX 3050 Laptop GPU, 16 GB of DDR4 RAM), in a development build connected to the Unity Profiler. During the measurements, the player stands still with a single weapon active, so the horde size stays stable. Profiler values come from a representative frame, and FPS values are read from the in-game FPS counter.

3.3.1 | 2,000 Enemies



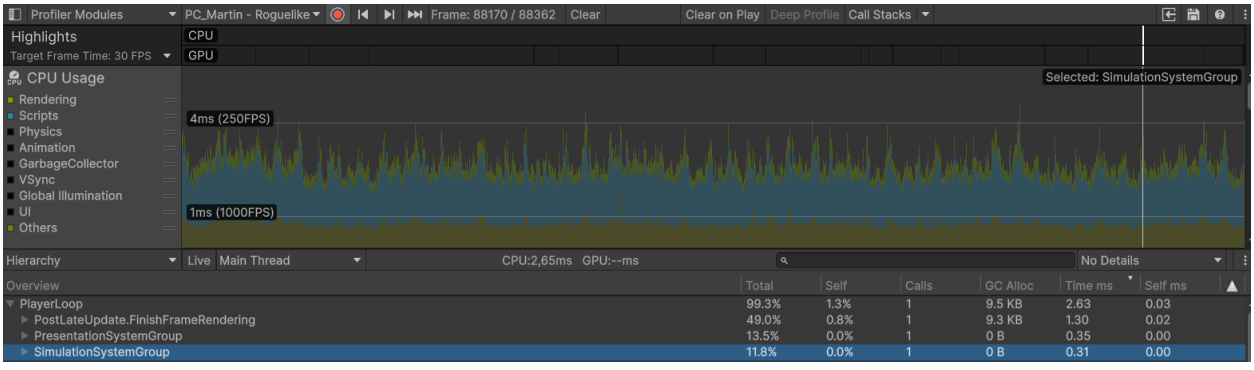
With 2,000 enemies, the simulation takes 0.24 ms, compared to 4.21 ms with the OOP implementation. The whole frame only takes 2.73 ms on the CPU.

3.3.2 | 4,000 Enemies



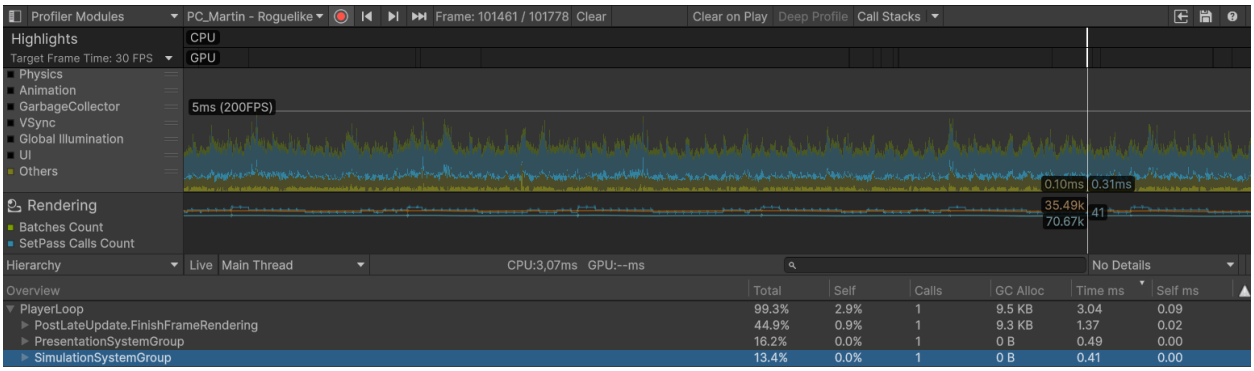
With 4,000 enemies, the simulation takes 0.29 ms (8.94 ms in OOP). Doubling the horde barely changes anything: at this scale, most of the cost is a fixed overhead (updating the systems, scheduling the jobs, synchronizations) rather than the enemies themselves.

3.3.3 | 6,000 Enemies



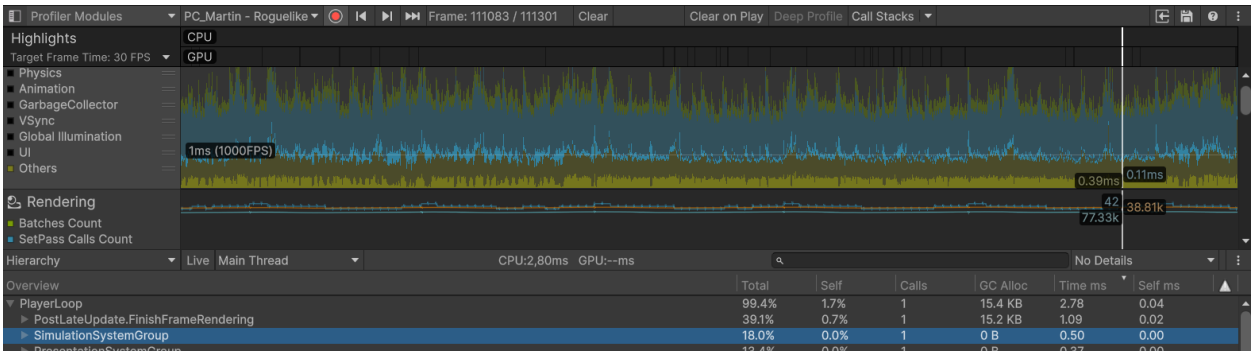
With 6,000 enemies, the simulation takes 0.31 ms (19.64 ms in OOP), while the OOP version was already down to around 60 FPS at this point.

3.3.4 | 8,000 Enemies



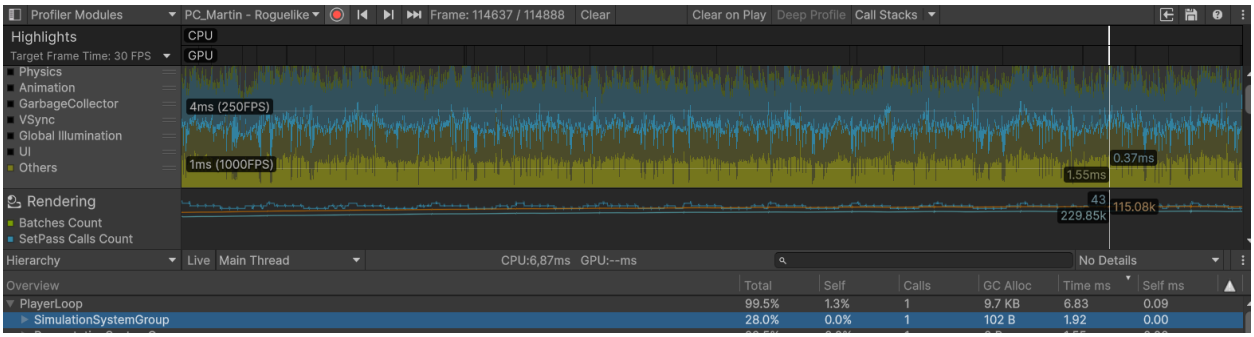
With 8,000 enemies, the simulation takes 0.41 ms (35.40 ms in OOP). Where the OOP implementation was barely playable at 30 FPS, the whole DOTS frame still only takes 3.04 ms on the CPU.

3.3.5 | 10,000 Enemies



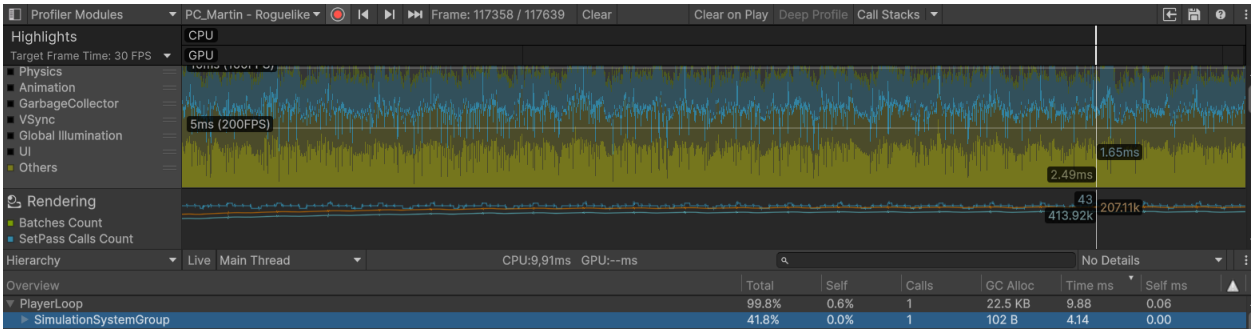
This is where the OOP implementation reached its limit: 45.59 ms for EnemyPool.Update() and below 20 FPS. With DOTS, the simulation of the same 10,000 enemies takes 0.50 ms, about 91 times faster, and the whole frame takes 2.78 ms. From this point on, there is no OOP equivalent anymore, so let's see how far the new implementation can go.

3.3.6 | 50,000 Enemies



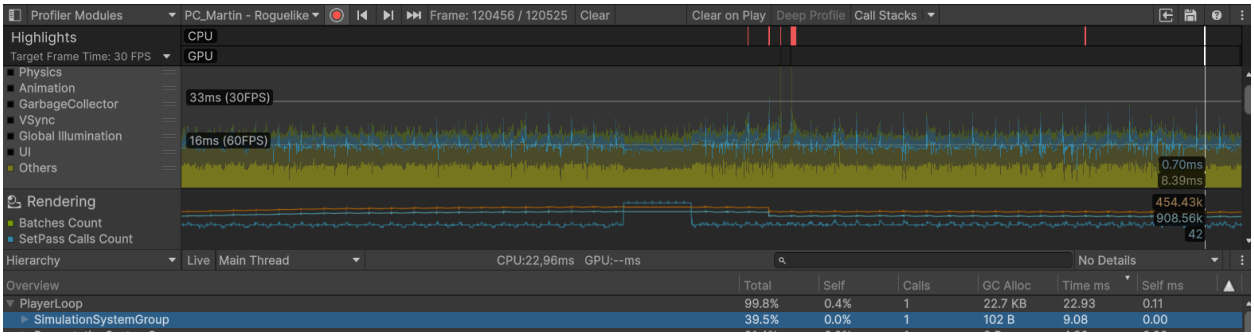
With 50,000 enemies, the simulation takes 1.92 ms and the whole frame 6.83 ms. The cost of each enemy now becomes visible: from here, the simulation time grows linearly with the horde size, at around 0.04 µs per enemy.

3.3.7 | 100,000 Enemies



With 100,000 enemies, ten times the OOP limit, the simulation takes 4.14 ms and the whole frame 9.88 ms. The game still runs at around 100 FPS.

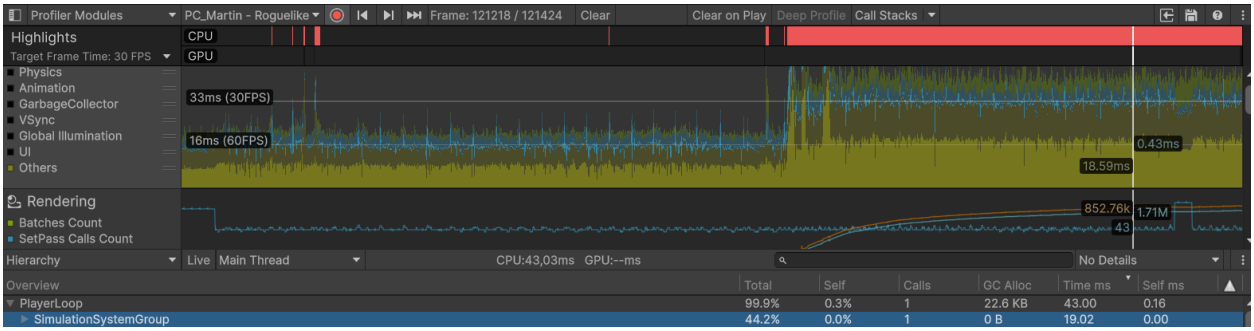
3.3.8 | 250,000 Enemies



With 250,000 enemies, the simulation takes 9.08 ms and the whole frame 22.93 ms, around 40 FPS. This is the last tested step that stays above the 30 FPS threshold, which I consider the practical limit of the

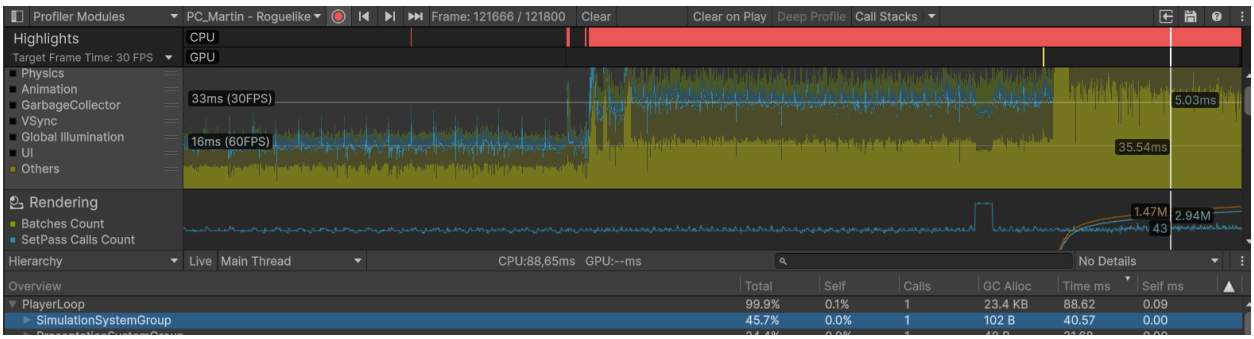
DOTS implementation.

3.3.9 | 500,000 Enemies



With 500,000 enemies, the simulation takes 19.02 ms and the whole frame 43.00 ms, around 20 FPS: the game drops below 30 FPS. The spikes visible at the beginning of the capture correspond to the spawning of the new enemies, which runs on the main thread in batches of up to 25,000 enemies per frame.

3.3.10 | 1,000,000 Enemies



With one million enemies, the simulation takes 40.57 ms and the whole frame 88.62 ms, around 10 FPS. The game is no longer playable, but the simulation of one million enemies still takes less time than the OOP implementation needed for 10,000 of them (45.59 ms).

3.4 Performance Summary table

Enemies	Simulation (ms)	Time per Enemy (µs)	Full CPU frame (ms)	In-game FPS
2,000	0.24	0.120	2.73	—
4,000	0.29	0.073	2.62	—
6,000	0.31	0.052	2.63	—
8,000	0.41	0.051	3.04	—

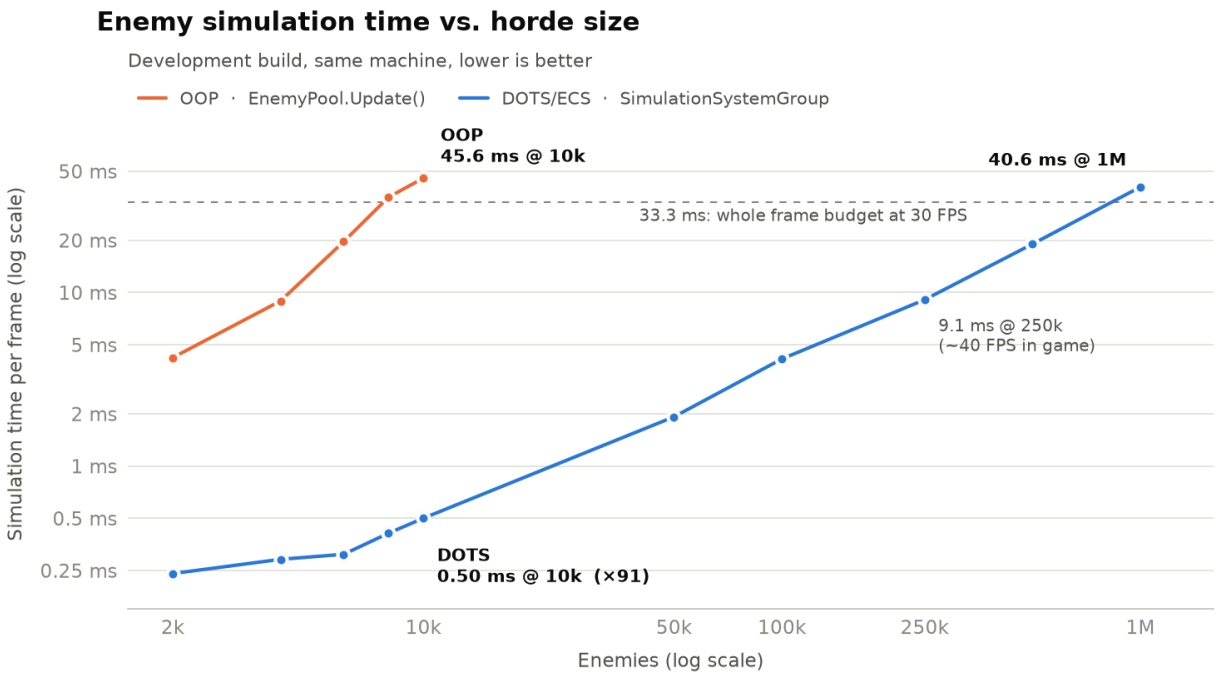
Enemies	Simulation (ms)	Time per Enemy (µs)	Full CPU frame (ms)	In-game FPS
10,000	0.50	0.050	2.78	~300
50,000	1.92	0.038	6.83	~160
100,000	4.14	0.041	9.88	~100
250,000	9.08	0.036	22.93	~40
500,000	19.02	0.038	43.00	~20
1,000,000	40.57	0.041	88.62	~10

Up to 10,000 enemies, the fixed overhead of the ECS pipeline dominates, which is why the first steps look almost flat. From 50,000 enemies onwards, the time per enemy stabilizes between 0.036 and 0.041 µs: the implementation scales linearly.

The FPS values were read from the in-game FPS counter (not recorded below 10,000 enemies). They stay slightly below what the CPU frame alone would allow (around 300 FPS observed at 10,000 enemies, for a 2.78 ms CPU frame): the difference comes from the render thread and the GPU, which the main-thread profiler does not show.

3.5 OOP vs DOTS/ECS Comparison

Enemies	OOP EnemyPool.Update() (ms)	DOTS SimulationSystemGroup (ms)	Speed-up
2,000	4.21	0.24	×17.5
4,000	8.94	0.29	×30.8
6,000	19.64	0.31	×63.4
8,000	35.40	0.41	×86.3
10,000	45.59	0.50	×91.2



Enemy simulation time vs. horde size (log-log scale)

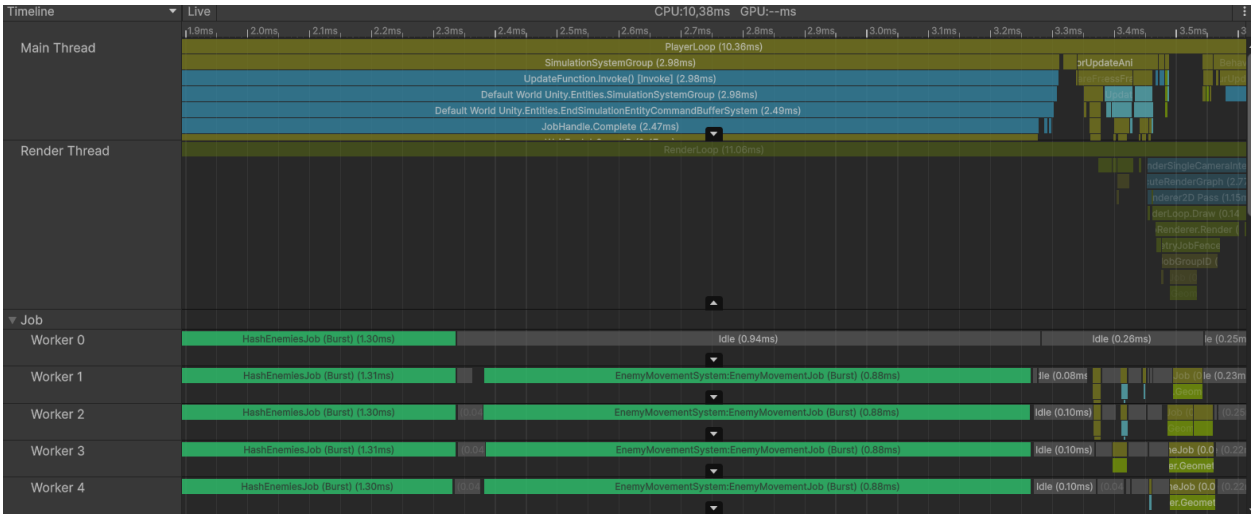
Two numbers summarize this comparison. Per enemy, the simulation cost drops from about 4.6 µs (OOP, 10,000 enemies) to about 0.04 µs (DOTS, 1,000,000 enemies), roughly 110 times less. On the whole CPU frame with 10,000 enemies, the game goes from 54.63 ms to 2.78 ms, about 20 times faster. The second number is smaller because the rest of the frame (rendering, UI, engine) did not benefit from the migration.

In practice, the playable limit moves from about 8,000 enemies at 30 FPS to about 250,000 enemies at around 40 FPS: around 30 times more enemies on screen.

These gains combine three effects that are not measured separately here: the contiguous memory layout, the Burst compilation and the multithreading. The rendering is also different (SpriteRenderer in OOP, Entities Graphics in DOTS), which contributes to the gain on the whole frame.

3.6 Multithreading in Action

The Timeline view of the Unity Profiler shows how the work is spread across the CPU cores. At 100,000 enemies:



Every worker thread runs the enemy jobs at the same time: HashEnemiesJob (about 1.30 ms) then EnemyMovementJob (about 0.88 ms), split across all the cores. In the OOP version, this whole workload ran sequentially on the main thread.

The main thread only waits: it schedules the jobs, then blocks in JobHandle.Complete (2.47 ms) inside the end-of-simulation command buffer system, which must wait for EnemyMovementJob before playing back its commands.

Building the spatial grid costs more than moving the enemies: 1.30 ms against 0.88 ms. Every enemy inserts itself into a single shared NativeParallelMultiHashMap, and these concurrent insertions most likely cost more than the steering math itself.

These two observations are the starting point of the optimization phase described in part 4.

3.7 Memory Management & Garbage Collection

The SimulationSystemGroup allocates 0 B per frame up to 10,000 enemies, and only 102 B above 50,000 enemies. All the ECS data lives in unmanaged memory (components and native containers), which the Garbage Collector never has to scan.

The allocations visible on the PlayerLoop (around 9.5 KB per frame with 2,000 enemies) almost entirely come from PostLateUpdate.FinishFrameRendering (9.3 KB), in other words from the render pipeline and not from the game simulation. As in the OOP phase, garbage collection is not a factor in these results.

3.8 Conclusion

The DOTS/ECS implementation broke the ceiling reached in the OOP phase. By storing the enemies contiguously in memory, compiling the logic with Burst and spreading it across all CPU cores, the

simulation of 10,000 enemies went from 45.59 ms to 0.50 ms, and the game now runs at around 40 FPS with 250,000 enemies.

This phase also moved the bottleneck. With a large horde, the simulation only represents around 40 to 45% of the CPU frame; the rest is spent on rendering and engine overhead. The profiler captures of this phase also revealed two weaknesses in the enemy pipeline itself, which the next part addresses.

Beyond the raw numbers, this project showed that DOTS is not a free performance switch: it requires designing around data, handling job dependencies and structural changes, and building bridges with the classic MonoBehaviour world. Used where it matters, on the thousands of identical entities of a Survivor-like, it turns a hard technical limit into a comfortable margin.

4. Profiling-Driven Optimization

4.1 Starting Point

At this point, the DOTS implementation was already far beyond the OOP limit. Instead of stopping there, I used the Timeline view of the profiler to look at what the CPU was really doing at 100,000 enemies, and optimized the bottlenecks it revealed, one step at a time. Each change was measured on its own, so that every gain, or absence of gain, can be attributed to a precise cause.

The Phase 2 Timeline (part 3.6) showed two problems:

A synchronization point: the main thread spent 2.47 ms per frame blocked in JobHandle.Complete. EnemyMovementJob records the enemies touching the player into the end-of-simulation command buffer, so this command buffer has to wait for the job before playing its commands back, and the main thread waits with it.

An expensive spatial grid: HashEnemiesJob (1.30 ms) cost more than EnemyMovementJob (0.88 ms).

4.2 Step A: Removing the Synchronization Point

Flags instead of commands: the movement job no longer records anything into a command buffer. It only enables two markers on the enemy, EnemyDeadTag and EnemyContactTag. Enabling a component is not a structural change, so it can be done safely from a parallel job, and from the main thread by the weapons.

One structural phase at the start of the frame: a new EnemyDeathSystem destroys all the flagged enemies in one batch, drops their experience orbs and damages the player, before any job is scheduled. The experience orb collection and the spawner also run there, and the few remaining commands go through the BeginSimulation command buffer, played back at the start of the next frame.

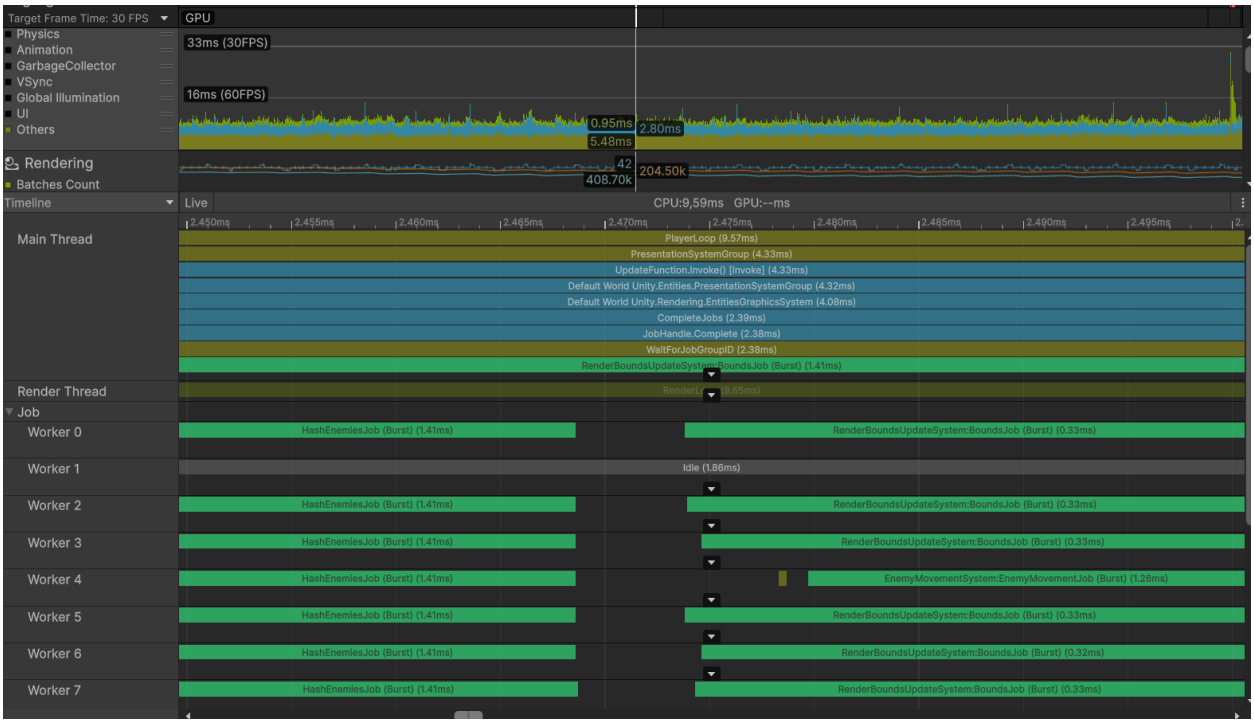
Result: nothing waits for the enemy jobs during the frame anymore. They keep running on the worker threads while the main thread prepares the rendering, and they are completed at the start of the next frame.

A new kind of bug: once jobs outlive their frame, every main-thread access to their data must complete them explicitly. The weapons read the spatial grid from MonoBehaviour.Update, at the start of the next frame: the hashing job had finished long before, but Unity's safety system still threw an exception because nothing had called Complete() on it. The grid now keeps the handle of the last job using it, and every main-thread reader completes it first, which costs nothing since the job is already finished.

Enemies	In-game FPS (Phase 2)	In-game FPS (Step A)
10,000	~300	~350
50,000	~160	~160

Enemies	In-game FPS (Phase 2)	In-game FPS (Step A)
100,000	~100	~100
250,000	~40	~43
500,000	~20	~22
1,000,000	~10	~11

The gain is real with small hordes (+17% FPS at 10,000 enemies), but becomes negligible from 100,000 enemies. The Timeline explains why:



The main thread no longer waits inside the simulation, but it now waits inside PresentationSystemGroup: 2.38 ms in JobHandle.Complete, for the rendering job RenderBoundsUpdateSystem:BoundsJob. This job could not start earlier, because all the worker threads were still busy with HashEnemiesJob. With every core already saturated, overlapping the simulation with the rendering only moved the wait.

Lesson: removing a synchronization point does not remove any work. It only helps when there are idle cores to fill. Once the workers are saturated, the only way to go faster is to give them less work.

A trap for the measurements: SimulationSystemGroup now only contains the scheduling of the jobs (between 0.2 and 4.5 ms from 10,000 to 1,000,000 enemies), so it can no longer be compared with the Phase 2 values. From this step on, I compare the frame time, the FPS and the job durations read in the Timeline.

4.3 Step B: A Sorted Spatial Grid

HashEnemiesJob was the most expensive job of the simulation. Every enemy inserts itself into a single NativeParallelMultiHashMap: all the threads write into the same structure, and the enemies of a cell end up scattered in memory, linked to each other like a list.

I replaced it with a counting sort, which stores the enemies contiguously, sorted by cell:

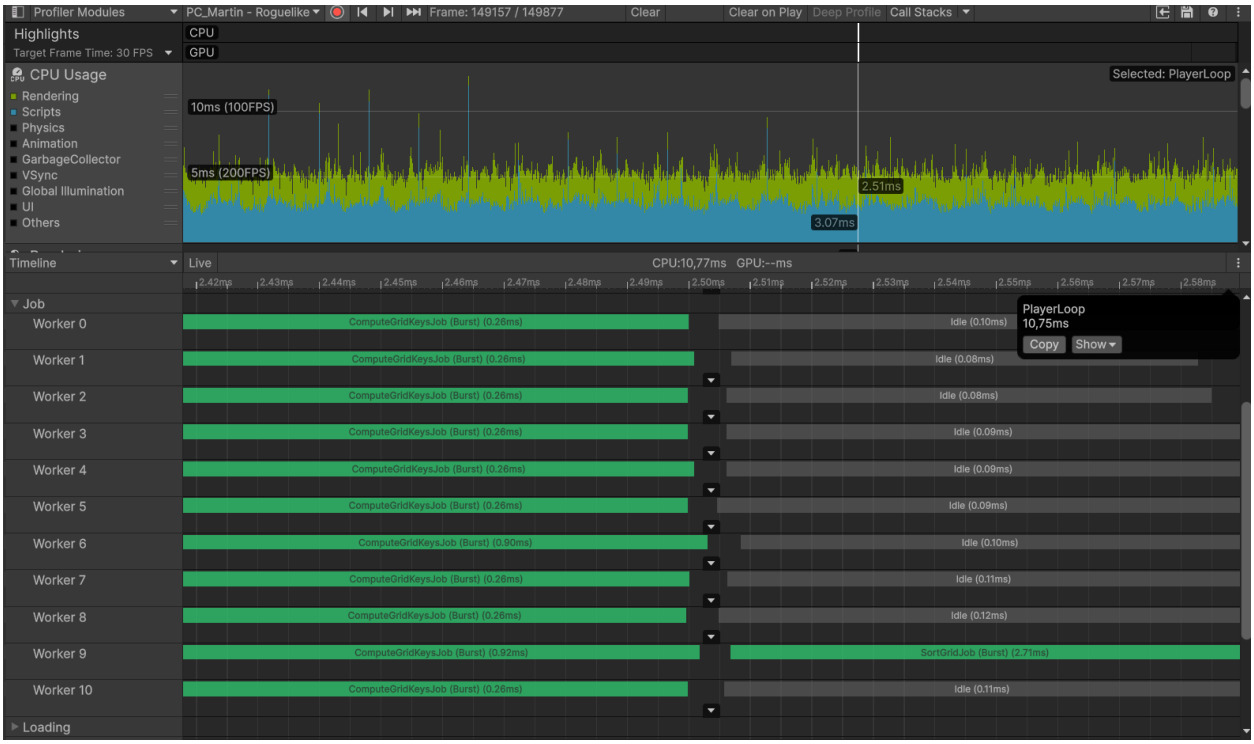
1. Keys: each enemy computes its cell and the matching bucket, and writes them at its own index in an array. No two threads ever write to the same place.

2. Offsets: the enemies are counted per bucket, and a prefix sum gives the first index of each bucket.

3. Scatter: each enemy is copied into its bucket, in a single contiguous array.

A neighbor query now reads a contiguous range of memory instead of following a linked list. Since several cells can share a bucket, each entry also keeps its cell, so lookups skip the enemies of other cells.

4.3.1 | First Version: a Sequential Sort



The keys job dropped to 0.26 ms, since it no longer has any contention, and EnemyMovementJob went from 1.26 ms to 0.95 ms thanks to the contiguous reads. The total CPU time spent on the grid was roughly halved, from about 11 ms (8 workers × 1.41 ms) to about 6 ms.

But the count, the prefix sum and the scatter ran in a single job, SortGridJob, on one thread for 2.71 ms, while the ten other workers were idle. The grid took about 3 ms from start to finish instead of 1.41 ms: by replacing parallel work with sequential work, this version lengthened the critical path.

4.3.2 | Second Version: a Parallel Sort

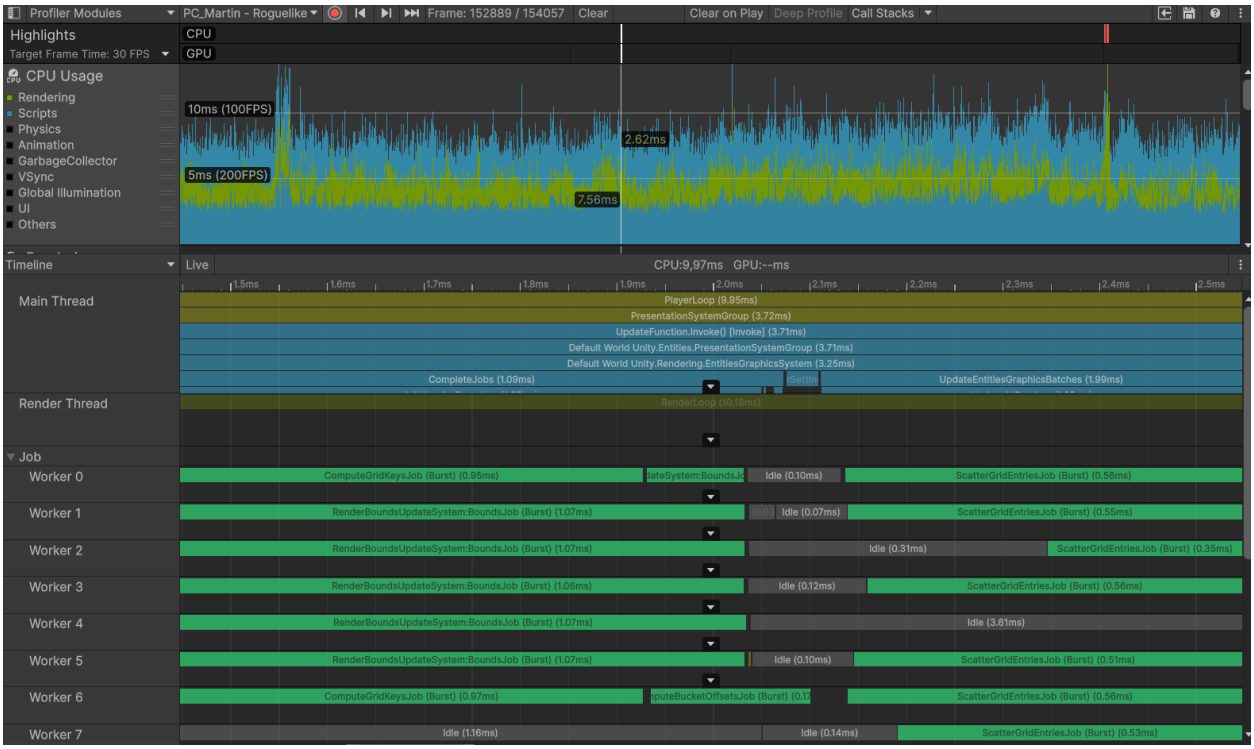
The counting and the scatter are actually parallel by nature:

Counting: the keys job atomically increments the counter of the enemy's bucket.

Prefix sum: the only sequential step, a single pass over the buckets (ComputeBucketOffsetsJob, 0.17 ms).

Scatter: each enemy atomically reserves the next slot of its bucket and copies itself there (ScatterGridEntriesJob, about 0.55 ms per worker).

Atomic operations were exactly what made the hash map slow, so why do they work here? Because they are spread over about 131,000 bucket counters: two threads very rarely touch the same counter at the same time, whereas all the hash map insertions go through shared counters.



The grid is now built in about 1.7 ms from start to finish, for about 6 ms of CPU time. In this capture, its jobs also interleave with the rendering jobs on the same workers: since nothing waits for the simulation anymore (step A), the workers simply execute whatever is ready.

Spatial grid	Start to finish	CPU time (all threads)	EnemyMovementJob
Hash map (Step A)	1.41 ms	~11 ms	1.26 ms
Sequential sort (B v1)	~3.0 ms	~6 ms	0.95 ms
Parallel sort (B v2)	~1.7 ms*	~6 ms	—

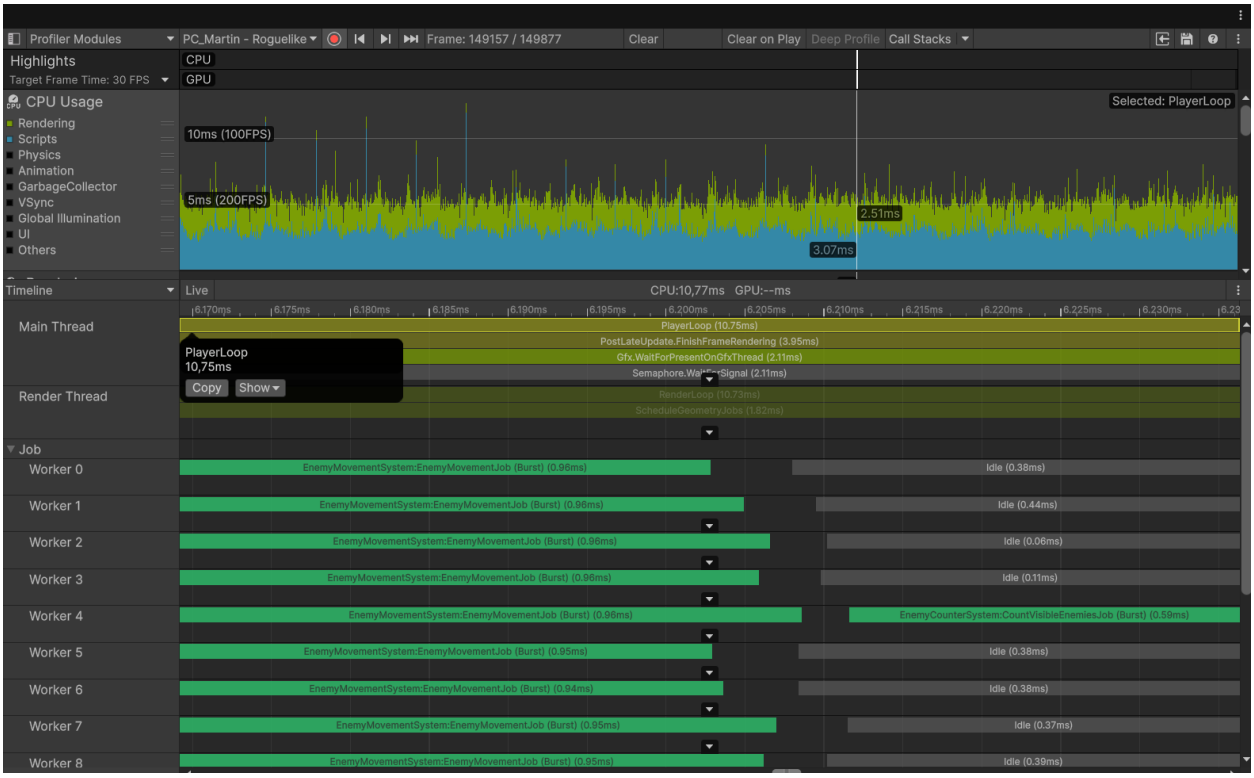
* While sharing the workers with the rendering jobs. All values measured at 100,000 enemies.

4.4 Results and the New Bottleneck

For this phase, I measured the frame time as a median over a selection of a few hundred frames (Highlights module of the profiler), which is more reliable than a single frame:

Enemies	Frame, Phase 2 (single frame)	Frame, Phase 3 (median)	In-game FPS (Phase 2 → 3)
50,000	6.83 ms	5.62 ms	~160 → ~160
100,000	9.88 ms	9.39 ms	~100 → ~100
250,000	22.93 ms	22.20 ms	~40 → ~43
500,000	43.00 ms	44.60 ms	~20 → ~22
1,000,000	88.62 ms	87.27 ms	~10 → ~11

Since the Phase 2 values come from a single frame, differences of about 1 ms are within the measurement noise: the frame time did not change significantly. The Timeline of the whole frame shows why:



The frame is bound by the render thread: RenderLoop takes 10.73 ms for a 10.75 ms frame, and the main thread ends its frame waiting for it (Gfx.WaitForPresentOnGfxThread, 2.11 ms). Meanwhile, the worker threads finish their work and become idle.

The most expensive job is now a rendering job: at every horde size, the most expensive marker of the profiler is ExecuteGpuUploads, from Entities Graphics. It uploads to the GPU the transform matrix of every enemy that moved, which means all of them, every frame: about 15 to 19 ms of CPU time per frame across all threads at 100,000 enemies, and up to 160 to 185 ms at 1,000,000.

This is Amdahl's law in practice: the simulation has been optimized until it became a small part of the frame, so further gains on it can no longer show in the framerate. The limit is now the rendering of hundreds of thousands of entities that all move every frame.

4.5 Conclusion

This optimization phase did not increase the framerate, but it reached its real goal: understanding where the time goes, and moving the bottleneck out of the game simulation. The enemy pipeline no longer blocks the main thread, its spatial grid costs about half as much CPU time, and the movement job is about 25% faster.

It also taught me three lessons that apply well beyond this project: removing a synchronization point is useless when the cores are already saturated; replacing parallel work with sequential work can lengthen the critical path even when the total work decreases; and every optimization has to be measured, because the intuitive result, here a higher framerate, is not always the one you get.

The next steps are on the rendering side: avoiding the upload of hundreds of thousands of matrices every frame, for example with a custom instanced renderer reading the enemy positions from a single GPU buffer. Moving the spawning logic into a Burst job would also remove the spikes when the horde grows.